Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Kısa tekrar ve yaratımsal kalıplara giriş
  - Önceki derste yazılım kalitesi, iyi tasarımın ölçütleri ve ilk örnek kalıp olarak MVC işlenmişti. MVC genel mimari düzeyde bir öneriydi; bu dersten itibaren daha ayrıntılı kalıplara, Dörtlü Çete (GoF) kitabındaki sırayla geçiliyor.
  - İlk kalıp, yaratımsal (creational) kalıpların ilki olan Abstract Factory'dir.
  - Kalıp adında "factory" kelimesi geçiyorsa o kalıp nesne üretmeye yöneliktir.
- Abstract Factory: amaç ve örnek problem (ekrandan, 00:10:00 ve 00:20:00)
  - Amaç: birbiriyle ilişkili ya da aynı aileden nesneleri, somut sınıflarını belirtmeden oluşturabilmek. Bunu mümkün kılan, kalıtımın getirdiği çok biçimliliktir.
  - GoF örneği: farklı görünüş ve davranış (look and feel) standartlarını ya da farklı pencere yöneticilerini (Motif, Presentation Manager; güncel karşılık olarak GNOME, KDE) destekleyen bir kullanıcı arayüzü takımı. Taşınabilirlik için istemci, hangi görünüşün hangi sınıftan nesnelerle gerçeklendiğini bilmemelidir.
  - Çözüm: her arayüz bileşeni için soyut bir üst sınıf (`Window`, `ScrollBar`) ve her pencere yöneticisi için somut alt sınıflar (`PMWindow`, `MotifWindow` vb.). Soyut `WidgetFactory` sınıfında `CreateScrollBar()` ve `CreateWindow()` soyut metotları vardır; `MotifWidgetFactory` ve `PMWidgetFactory` bunları gerçekler.
  - İstemci yalnız soyut fabrika ve soyut ürünlerle konuşur. Hangi somut fabrika verilirse o ailenin nesneleri üretilir; aile çalışma anında değiştirilebilir.
- Abstract Factory kalıp yapısı (ekrandan, 00:25:40)
  - `AbstractFactory` (`CreateProductA()`, `CreateProductB()`), `ConcreteFactory1` ve `ConcreteFactory2`, `AbstractProductA` / `AbstractProductB` ve bunların aile üyeleri (`ProductA1`, `ProductA2`, `ProductB1`, `ProductB2`), `Client`.
  - Soyut fabrikada üye alan yoksa soyut sınıf yerine arayüz (interface) kullanılabilir. Hocanın önerisi: gerek yoksa her zaman arayüzü tercih edin.
- Abstract Factory'nin uygulanabileceği durumlar (ekrandan, 00:30:00)
  - Bir sistemin, kendi ürünlerinin nasıl oluşturulduğundan, birleştirildiğinden ve sunulduğundan bağımsız olması istendiğinde.
  - Bir sistemin farklı ailelerden herhangi birinin nesneleriyle yapılandırılması gerektiğinde.
  - İlişkili ürün ailelerinden oluşan nesnelerin birlikte kullanılması gerektiğinde.
  - Bir ürün ailesinin sınıf kütüphanesi olarak sunulacağı, ama gerçeklemeleri yerine yalnız arayüzlerinin açığa çıkarılacağı durumlarda.
- UML gösterimi üzerine düzeltmeler (ekrandan, 00:10:00)
  - GoF kitabı yayımlandığında UML standardı henüz oturmamıştı. Kitaptaki kalıtım okları, üçgenden çok ER diyagramlarındaki "kaz ayağına" benzer ve ok yönü eskidir. Doğru çizimde ok ucu (içi boş üçgen) üst sınıfa bakar.
  - İtalik yazım soyutluğu gösterir, ama elle çizimde zordur. Soyut sınıf adının üstüne `<<abstract>>` stereotipi, soyut metodun imzasının sonuna `{abstract}` yazılır. `+` public, `-` private demektir.
  - Kesik çizgili, ucu açık ok kullanma (dependency) ilişkisidir: bir sınıfta diğer türden üye alan yoksa ama bir metodun parametresinde ya da dönüş türünde o tür geçiyorsa kullanılır. GoF şemalarındaki kesik çizgiler doğrudur.
  - Elmas, parça-bütün ilişkisidir; elmas bütün tarafına konur.
- Abstract Factory örneği: üç boyutlu oyun motoru (hocanın kendi örneği) (ekrandan,

00:40:00, 00:50:00, 00:52:30)
  - Oyun, işletim sistemine ve ekran kartına göre OpenGL ya da DirectX kütüphanesiyle çalışacak. Renderer çizimi, shader gölgelemeyi yapan bileşendir.
  - Çözüm 1 (switch-case): `ApControl` sınıfında `LibraryType` enum alanı ve dört somut üye (`OpenGLRenderer`, `OpenGLShader`, `DirectXRenderer`, `DirectXShader`) vardır. `doRendering` ve `doShading` metotlarının her birinde ayrı bir switch bulunur.
  - Çözümün zayıf yönü: yeni bir kütüphane (ör. DirectX 10, 11, 12) eklemek için enum'a, üye alanlara ve her switch'e dokunmak gerekir. Yeni bir işlem (ray tracing, texture mapping) eklemek ise her aile için yeni sınıf ve yeni case demektir.
  - Tasarım ipucu (slayt): switch-case kullanımı, çok biçimliliğe gereksinim duyulduğunun ve/veya sorumlulukların atanmasında yanlışlık olduğunun uyarı işaretidir.
  - Çözüm 2 (doğru kalıtım): soyut `Shader` ve `Renderer` üst sınıfları, altında OpenGL ve DirectX somut sınıfları. `ApControl` yalnız bir `Renderer` ve bir `Shader` tutar. Switch yalnız kurucuda bir kez kalır; `doRendering` ve `doShading` doğrudan soyut tür üzerinden çağrı yapar. Yeni kütüphane için yalnız kardeş sınıflar (`DX12Shader`, `DX12Renderer`) eklenir, mevcut kod değişmez.
  - Çözüm 3 (Abstract Factory): soyut `ComponentFactory` (`createRenderer()`, `createShader()`) ve somut `OpenGLFactory`, `DirectXFactory`. `library` alanı artık enum değil fabrikadır. Yeni kütüphane için ürün sınıflarına ek olarak bir somut fabrika da yazılır.
  - Değerlendirme: bu örnekte asıl kazanç doğru kalıtımdan gelir. Kalıp, yapılandırmayı biraz rahatlatır ama yeni sınıflar ve yeni bağlaşımlar getirir. Hocaya göre switch-case dışındaki iki çözümden hangisi seçilse uygundur.
- Örnekten çıkarılan tasarım kuralları (ekrandan, 01:00:00 ve 01:20:00)
  - Neyin değiştiğini bul ve onu sarmala: değişen şey ayrı bir sınıf ya da sınıf hiyerarşisine konur; böylece soyutlanır, kara kutu gibi kullanılır ve o konudaki değişiklikler yalnız o kısmı etkiler. Örnekte değişen şey grafik kütüphanesiydi ve `ComponentFactory` ile soyutlandı.
  - Kalıtım ilişkisi yerine parça-bütün ilişkisini tercih et: kalıtım sınıf düzeyinde ve kalıcıdır, çalışma anında değiştirilemez; parça-bütün ilişkisi ise yeni bir nesne atanarak değiştirilebilir.
  - Gerçeklemelere göre değil, arayüzlere göre tasarla: `ApControl`, fabrikadan bir renderer ve shader ister; bunların nasıl oluşturulduğu umurunda değildir.
- Yanlış kurulmuş kalıtım örneği (ekrandan, 01:30:00)
  - Soyut `ApControl` altından `OpenGL_ApControl`, `DirectX_ApControl`, `DirectX12_ApControl` türetilseydi oyun mantığının tamamı her alt sınıfta tekrarlanırdı (duplicate code). Her yeni kütüphane ya da yeni işlem (ör. `OpenGLTextureMapper`) çok daha fazla iş getirirdi.
  - Ders: kalıtım hiyerarşisini kurarken değişen şeyi hiyerarşiye koy.
- Factory Method: amaç ve örnek problem (ekrandan, 01:40:00)
  - Amaç: nesne oluşturmak için öyle bir arayüz sunmak ki oluşturulacak nesnenin sınıfına bu arayüzü sağlayan sınıfın alt sınıfları karar verebilsin.
  - Örnek: farklı belge türleriyle çalışan uygulamalar için bir çatı (framework). Çatı, kullanıcılarının üreteceği yeni belge türlerini önceden bilemez.
  - Çözüm: soyut `Application` (`CreateDocument()` soyut; `NewDocument()` ve `OpenDocument()` somut) ve soyut `Document` (`Open()`, `Close()` soyut; `Save()`, `Revert()` somut). `NewDocument()` içinde `doc = CreateDocument(); docs.Add(doc); doc->Open();` çalışır. Kullanıcı `MyApplication` ve `MyDocument` alt sınıflarını yazar; `CreateDocument()` yalnız `return new MyDocument` der.
  - Bu sınıflarda somut metotlar da olduğu için arayüz kullanılamaz, soyut sınıf gerekir.
  - Java'daki generic sınıflarla da benzer bir çözüm kurulabilir, ama hoca dile özgü olduğu için bu yolu tercih etmiyor.
- Factory Method kalıp yapısı ve uygulanabileceği durumlar (ekrandan, 01:46:00 ve 01:50:00)
  - `Creator` (soyut `FactoryMethod()`, somut `AnOperation()` içinde `product = FactoryMethod()`), `ConcreteCreator` (`return new ConcreteProduct`), `Product`, `ConcreteProduct`.
  - Bir sınıf oluşturacağı nesnelerin türünü önceden bilemiyorsa ya da bir sorumluluğu yardımcı alt sınıflara devrediyorsa ve hangi yardımcının oluşturulacağına karar vermek gerekiyorsa kullanılır.
  - Paralel hiyerarşi örneği: `Figure` (`CreateManipulator()`) ve `Manipulator` (`DownClick()`,

`Drag()`, `UpClick()`); `LineFigure` bir `LineManipulator`, `TextFigure` bir `TextManipulator` oluşturur. Bazı figürler ortak bir manipülatör paylaşabilir; özel manipülatöre ihtiyaç duymayan şekiller için varsayılan bir manipülatör eklenebilir ya da üst sınıf somut yapılabilir.

- Factory Method örneği: e-ticaret siparişleri (Head First Design Patterns'ten) (ekrandan, 02:00:00, 02:04:30, 02:07:40)
  - Sipariş adımlarının sırası sabittir, ama her adımın iş mantığı ürün türüne göre çok farklıdır (elektronikte garanti, kırılacak eşyada özel paketleme, kitapta farklı vergi vb.).
  - Soyut `Store` (`# createOrder(uniqueID): Order` soyut, `+ makeOrder()` somut) ve soyut `Order` (`orderFromSupplier()`, `pack()`, `prepare()`, `sendToCustomer()` hepsi soyut). `ElectronicsStore` → `Electronics`, `BookStore` → `Books` nesnesi oluşturur.
  - `makeOrder()` önce `createOrder()` ile siparişi oluşturur, sonra `prepare()`, `orderFromSupplier()`, `pack()`, `sendToCustomer()` adımlarını sırayla çağırır.
- Kalıp adlarına dikkat
  - Adı, amacı ya da UML şeması birbirine benzeyen kalıplar vardır ama aynı değildirler; doğru kalıbı seçmek için farkları bilinmelidir.
  - "Factory kalıbı" diye bir kalıp yoktur. Abstract Factory ve Factory Method vardır; "factory" nesne üreten metotlara verilen genel addır.
- Builder: amaç ve örnek problem (ekrandan, 02:30:00)
  - Amaç: karmaşık bir nesnenin oluşturulmasını gösteriminden ayırmak; böylece aynı oluşturma süreciyle farklı gösterimler elde etmek.
  - GoF örneği: RTF okuyucu. Belge düz metne (ASCII), TeX'e ya da düzenlenebilir bir metin bileşenine (TextWidget) çevrilebilmeli; ileride yeni biçim eklemek kolay olmalı.
  - Çözüm: `RTFReader.ParseRTF()` her token'ı okur ve türüne göre (CHAR, FONT, PARA) builder'ın `ConvertCharacter`, `ConvertFontChange` ya da `ConvertParagraph` metodunu çağırır. Soyut `TextConverter` altında `ASCIIConverter`, `TeXConverter`, `TextWidgetConverter`; her biri kendi sonucunu (`GetASCIIText()`, `GetTeXText()`, `GetTextWidget()`) verir.
  - Slayttaki eleştiriler: çok biçimlilik tam kullanılamadı, switch-case hâlâ duruyor; alt sınıflar üst sınıfın tüm metotlarını gerçeklemek zorunda olduğundan anlamsız metotlar boş gövdeli kalıyor (ör. `ASCIIConverter` için font ve paragraf).
- Builder kalıp yapısı ve uygulanabileceği durumlar (ekrandan, 02:40:00)
  - `Director.Construct()` yapıdaki her nesne için `builder->BuildPart()` çağırır; `Builder` (soyut `BuildPart()`), `ConcreteBuilder` (`BuildPart()`, `GetResult()`), `Product`.
  - Şemada switch görünmez, ama parçaların `BuildPart`'a aktarılması sırasında "parça şu ise şunu yap" mantığı bir yerde ortaya çıkar.
  - Karmaşık bir nesneyi oluşturma algoritması hem parçalardan hem de parçaların nasıl birleştirileceğinden bağımsız olmalıysa ya da oluşturma süreci farklı gösterimlere izin vermeliyse kullanılır.
  - Ürünün iç yapısı yeni bir ConcreteBuilder yazılarak az yan etkiyle değiştirilebilir. Yeni bir parça türü eklenirse switch'e yeni case ve ilgili build metodu eklemek gerekir; hocaya göre bu bedel kabul edilebilir.
- Sequence diyagramlarında dönüş okları
  - Hoca, Builder'ın etkileşim şemasında (ve birçok kitapta) dönüş oklarının çizilmediğini eleştirdi. Dönüş okları olmadan bir metodun diğerinin içinde mi, yoksa bittikten sonra mı çağrıldığı anlaşılamaz. Hoca dönüş oklarının mutlaka çizilmesinden yana.
- Builder ile Abstract Factory / Factory Method farkı
  - Builder'da istemci (Director) ürünün oluşturulma süreci üzerinde çok daha fazla denetim sahibidir: parçaların sırasını, birleştirilmesini ve araya girecek işleri yönetir.
  - Abstract Factory ve Factory Method'da ise "şu lazım" denir ve ürün alınır; oluşturmanın ayrıntılarına karışılamaz.
- Builder örneği: çok markalı araç bayisi (hocanın kendi örneği) (ekrandan, 02:50:00, 02:55:00, 03:00:00, 03:03:25)
  - Her marka model adında farklı kısaltmalar kullanır. Araçlar kasa, yakıt türü, motor hacmi ve gücü gibi bileşenlerden karmaşık bir süreçle oluşur.
  - Ürün paketi: `Car` (`brand`, `model`), `Engine` (`volume`, `power`), `Transmission` (`gearCount`, `isAutomatic`, ikisi de `final`), `Chassis` enum (`SEDAN, HATCHBACK, STATIONWAGON, SUV`), `FuelType` enum (`PETROLEUM, DIESEL, LPG, ELECTRIC`).
  - Abstract Factory burada uygun değildir: o kalıpta ürün aileleri ortaktır, burada ise

ürün çok sayıda alt parçadan oluşur ve bu parçalar markadan markaya farklı yorumlanır.
  - `CarDirector` bir `CarBuilder` ile çalışır; `construct()` her elemanı sekmeyle ayırıp "Engine" ise `buildEngine`, "Gear" ise `buildTransmission` çağırır. `HondaBuilder` ve `RenaultBuilder` markaya özgü yorumlamayı yapar.
  - `getResult()` public'tir; ürün tamamlanmamışsa vermez, tamamsa ürünün kopyasını döndürür. Alt sınıflar ürüne protected bir getter üzerinden erişir.
  - Çıktı: `Car: Honda Civic Sedan (Sedan)`, `Engine: Petroleum 1600cc 125HP`, `Transmission: 6 gears automatic transmission`.

Hocanın Özellikle Vurguladığı Kısımlar

- UML sınıf şemalarında güncel gösterim
  - GoF kitabındaki eski kalıtım gösterimi kullanılmamalıdır. Ok ucu üst sınıfa bakan içi boş üçgen, soyut sınıf için `<<abstract>>`, soyut metot için `{abstract}`, metotlarda + işareti kullanılmalıdır. Kâğıt üstünde italik belli olmadığı için bu gösterim daha uygundur.
- Switch-case bir uyarı işaretidir
  - Kod yazarken switch-case kullandığınızı görüyorsanız büyük ihtimalle orada kalıtım ve çok biçimlilikle daha iyi bir tasarım yapılmalıdır. Switch içeren metotlar da uzar.
  - Ancak iyi tasarım switch'i tamamen ortadan kaldırmayabilir; önemli olan onu "önemli yerlerden" kaldırmaktır. Hangi ailenin kullanılacağına bir yerde bir kez karar verilmesi gerekir.
- Aşırı esneklik de zarardır
  - Gelecekteki bütün değişiklikler öngörülemez. Kısa vadede olası değişiklikleri göz önünde bulundurarak tasarım yapmak en mantıklısıdır.
- Kalıbın ilk maliyeti kısa sürede amorti olur
  - Kalıpla yapılan çözüm ilk akla gelen çözümden biraz daha fazla iş gerektirse de bu iş kısa zamanda fazlasıyla geri kazanılır.
- Benzer kalıpların farkını bilin
  - Doğru kalıbı seçebilmek için benzer kalıpların (Abstract Factory, Factory Method, Builder) farkları bilinmelidir. "Factory kalıbı" demek yanlıştır; hangi kalıbın kullanılacağı açıkça belirtilmelidir.
- Kalıbın ne zaman uygulanacağı ezberlenmemeli, anlaşılmalı
  - Konuşma dili belirsizlik taşır; "uygulanabileceği durumlar" maddeleri kelime kelime ezberlenmemeli, temel fikir kavranmalıdır.
- Kalıtım güçlü bir araçtır ama dikkat ister
  - Kalıtımın yanlış kullanımı çok ters sonuçlar doğurur. Kalıtım hiyerarşisi içermeyen kalıp sayısı bir elin parmaklarını geçmez.
- Builder'daki boş gövdeli metotlar ciddi bir sorundur
  - Alt sınıf, üst sınıfın daha özelleşmiş halidir; üst sınıfta olan her şey alt sınıfta anlamlı olmalıdır. Boş gövdeli metotlar kalıtımın çekirdek tanımına aykırıdır ve kod kalitesi politikalarını ihlal edebilir.

Kısa Tekrar Notları

- Adında "factory" geçen kalıp nesne üretimine yöneliktir. Yaratımsal kalıplar: Abstract Factory, Factory Method, Builder (bu derste).
- Abstract Factory: ilişkili nesne ailelerini somut sınıf belirtmeden üretir. Soyut fabrika + soyut ürünler; istemci yalnız soyutları görür.
- Soyut fabrikada üye alan yoksa arayüz tercih edilir.
- Oyun motoru örneği: switch-case çözümü → doğru kalıtım çözümü → Abstract Factory çözümü. Asıl kazanç doğru kalıtımdan gelir.
- Üç tasarım kuralı: neyin değiştiğini bul ve sarmala; kalıtım yerine parça-bütün ilişkisini tercih et; arayüzlere göre tasarla.
- Yanlış kalıtım (`ApControl` alt sınıfları) kod tekrarına yol açar.
- Factory Method: nesne oluşturma kararını alt sınıfa bırakır. `Creator.AnOperation()` içinde `FactoryMethod()` çağrılır; `ConcreteCreator` hangi ürünün üretileceğini belirler.

- Factory Method örnekleri: `Application/Document` çatısı, `Figure/Manipulator` paralel hiyerarşisi, `Store/Order` e-ticaret örneği.
- Builder: karmaşık nesnenin oluşturulmasını gösteriminden ayırır. `Director` parçaları sırayla `Builder`'a yaptırır; sonuç `GetResult()` ile alınır.
- Builder eleştirileri: switch-case kalır; alt sınıflarda anlamsız, boş gövdeli metotlar olabilir.
- Builder'da istemci oluşturma süreci üzerinde ayrıntılı denetime sahiptir; Abstract Factory ve Factory Method'da değildir.
- Kesik çizgili ucu açık ok = kullanma (dependency); elmas = parça-bütün (elmas bütün tarafında).
- Sequence diyagramlarında dönüş okları çizilmelidir.

Detaylı Açıklamalar

- Abstract Factory'nin özü, iki paralel soyutlamadır: bir fabrika hiyerarşisi ve bir ürün hiyerarşisi. İstemci, `WidgetFactory` gibi soyut bir fabrikaya `CreateWindow()` dediğinde, elindeki nesne aslında `MotifWidgetFactory` ise dönen pencere `MotifWindow` olur. İstemci bunu bilmez; yalnız `Window` türünü görür. Böylece en özelleşmiş sınıf türünü belirtmeden belirli bir türden nesne oluşturulmuş olur. Fabrika nesnesini değiştirmek, bütün ailenin değişmesi demektir.
- Oyun motoru örneği, bir kalıbın değerini üç aşamada gösterir. İlk çözümde `ApControl` dört ayrı üye tutar ve her işlemde (`doRendering`, `doShading`) kütüphaneye göre switch yapar. Yeni bir DirectX sürümü geldiğinde enum'a değer eklenir, iki yeni sınıf yazılır, iki yeni üye alan tanımlanır ve her switch'e yeni case eklenir. Hoca, küçük bir değişiklik için kodun birçok yerine dokunmayı gerektiren bu durumu ileride görülecek "shotgun surgery" ve "divergent change" kod kusurlarıyla ilişkilendirdi; ikisi de bakım kolaylığının olmadığını gösterir.

  Ek bilgi (derste anlatılmadı): Fowler'ın tanımlarında iki kusur büyük olasılıkla şöyle ayrılır: divergent change, tek bir sınıfın farklı nedenlerle sürekli değiştirilmesidir; shotgun surgery ise tek bir değişikliğin birçok sınıfta küçük düzeltmeler gerektirmesidir. Oyun motoru örneğinde yeni kütüphane eklemek hem `ApControl` içinde birçok yere hem de yeni sınıflara dokunmayı gerektirdiği için iki kusurun belirtileri birlikte görülür.

- İkinci çözümde soyut `Renderer` ve `Shader` sınıfları kurulur. `ApControl` artık yalnız bir renderer ve bir shader tutar; kurucuda bir kez switch ile hangi somut nesnelerin verileceği belirlenir, sonra bütün işlemler soyut tür üzerinden yürür (ekrandan, 00:50:00):

```
package dp.abstractFactory.solution2;
public class ApControl {
    private LibraryType library;
    private Renderer r;
    private Shader s;
    //library nesnesini ilklendirip kütüphane belirle
    public ApControl( ) {
        switch (library){
            case OpenGL:
                r = new OpenGLRenderer();
                s = new OpenGLShader();
                break;
            case DirectX:
                r = new DirectXRenderer();
                s = new DirectXShader();
                break;
        }
    }
    void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    void doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
```

```
    }
}
```

- Bu çözümde yeni kütüphane eklemek yalnız yeni kardeş sınıflar yazmaktır; mevcut koda dokunulmaz. Bu, dağıtılmış koda müdahale etmeden yalnız ekleme yapılarak güncelleme yapılabilmesi demektir. Üçüncü çözüm (Abstract Factory) buna ek olarak bir fabrika hiyerarşisi getirir: `library` alanı `ComponentFactory` türündedir ve renderer/shader `createRenderer()`/`createShader()` ile alınır. Hoca, fabrikanın kurucuya parametre olarak verilebileceğini ve renderer ile shader'ı ayrı ayrı tutmak yerine yalnız fabrikayı üye olarak tutmanın da mümkün olduğunu gösterdi. Değişiklik maliyeti açısından ikinci ve üçüncü çözüm aynı düzeydedir; kalıp yalnız yapılandırma rahatlığı getirir.
- "Kalıtım yerine parça-bütün" kuralı kalıtımı yasaklamaz. Kalıtım sınıf düzeyinde kurulan kalıcı bir ilişkidir: bir alt sınıf nesnesini kardeş sınıfa çevirmek gerektiğinde yeni bir nesne oluşturup bilgileri kopyalamak gerekir. Parça-bütün ilişkisi ise çalışma anında yeni bir nesne atanarak değiştirilebilir. Deneyimli tasarımcılar, bir çözüm kalıtımla da kalıtımsız da yapılabiliyorsa kalıtımsız olanı seçme eğilimindedir.
- Factory Method'un Abstract Factory'den farkı, oluşturma kararının bir fabrika nesnesine değil alt sınıftaki tek bir metoda bırakılmasıdır. Çatı (framework) örneğinde `Application.NewDocument()` belge türünden bağımsız işleri (belgeyi listeye ekleme, açma) somut olarak yapar ve yalnız belgeyi oluşturma işini soyut `CreateDocument()`'e bırakır. Çatının kullanıcısı yalnız bu metodu gerçekler. Bir çatıya ne kadar somut metot konabilirse kullanıcının yükü o kadar azalır.
- E-ticaret örneğinde `makeOrder()`, siparişin sabit adım sırasını tutan somut metottur; adımların içeriği `Order` alt sınıflarındadır (ekrandan, 02:07:40):

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
        newOrder.pack( );
        newOrder.sendToCustomer( );
    }
}
public class ElectronicsStore extends Store {
    protected Order createOrder( String uniqueID ) {
        Electronics newElectronicsOrder = new Electronics(uniqueID);
        //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
        return newElectronicsOrder;
    }
}
```

- Builder, ürünün parça parça kurulduğu durumlar içindir. Director süreci yönetir, ConcreteBuilder parçaların nasıl yorumlanacağını bilir. Araç bayisi örneğinde Honda'nın model metninde "ECO" geçmesi LPG, "PS" geçmesi motor gücü, "L" geçmesi litre cinsinden hacim demektir; `1.6L` değeri `(int)(Double.parseDouble("1.6") * 1000)` ile 1600 cc'ye çevrilir (ekrandan, 03:00:00). `main` metodunda `HondaBuilder("Civic Sedan")` ve `{"Engine\t125PS", "Engine\t1.6L", "Gear\t6AT"}` verilir; kasa bilgisi verilmediği için varsayılan Sedan seçilir (ekrandan, 03:03:25).
- Builder'ın iki zayıf yönü vardır. Birincisi, parçaların türüne göre dallanma (switch ya da if-else) Director'da kalır; bu kaçınılmazdır, çünkü belge ya da model metni farklı türde parçalar içerir. İkincisi, soyut builder'daki her metot her somut builder için anlamlı olmayabilir; bu durumda alt sınıfta boş gövdeli metotlar yazılır ve bu kalıtımın tanımına aykırıdır. Hoca, böyle bir çalışma biçimine gerçekten ihtiyaç varsa "her güzelin bir kusuru vardır" diyerek kalıbın yine de seçilebileceğini söyledi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.